

PA3 report

Name: Mustafa Özen
Id: 32679

Introduction:

In the third programming assignment we were given a task where there are threads (referred as visitors) and we need to synchronize the threads so that they can fulfill the requirements of entering a tour. To achieve atomicity and minimize the overhead and maximize parallelization I used semaphores and conditional variables and mutexes. The only purpose of mutex is to achieve atomicity during updating the variables in the class.

Tour class:

The implementation of the tour class is as follows:

```
class Tour {
public:
    // Constructor
    Tour(int visitorCount, int guide_required);

    // Destructor
    ~Tour();

    // Methods
    void arrive(); // Called when a visitor arrives
    void start(); // Placeholder, already defined in the test files
    void leave(); // Called when a visitor leaves

private:
    // Shared state variables
    //int total_num; // Total number of threads
    int visitor_count; // Number of visitors in a tour, excluding the guide
    int all_count; // Number of visitors in a tour, including the guide
    //int num_tours; // Number of tours completed
    int current_visitors; // Current number of visitors at the site
    bool tour_in_progress; // True if a tour is ongoing
```

```

bool has_guide;    // Indicates if a guide is needed for the tour

// Synchronization primitives
pthread_mutex_t visitor_mutex;    // Protect shared variables(used)
//sem_t semaphore_arrive;        // Block new arrivals during a tour(no need)
sem_t semaphore_tour;            // Allow visitors to start the tour(used)
//pthread_barrier_t barrier_tour_start; // Synchronize tour start(no need)
pthread_t guide_thread;          // Track the guide thread (if any)(used)
pthread_cond_t cond_tour_end;    //used
};

```

The constructor expects 2 variables: visitorCount and guide_required. VisitorCount variable refers to how many visitors there will be in the tour. Guide_required is an int variable to hold whether a guide exists in a tour or not. The implementation will be explained later.

We declare a destructor to free back the memory we allocated, the implementation will be explained later.

There are 3 functions in total, and we only implement 2 of them which are the arrive() and leave() functions. The start() function is just declared in Tour.h because the real implementation will be implemented in the main function.

There are private member variables to help achieve this concurrent task, we keep the **visitor_count** which refers to number of visitors in a tour, excluding the guide. **All_count** variable is the updated version of visitor_count which handles the logic when there is a guide in the tour, in that case we add 1 to total number of visitors in a tour. **Current_visitors** refer to the current number of visitors at the site, this will be used to keep track of the amount of visitors that have entered the tour site. **Tour_in_progress** variable notifies other threads whether there was an ongoing tour at the site. **Has_guide** variable tells us whether there must be a guide in the tour or not. **Visitor_mutex** variable is used to protect shared variables, this lock is used to achieve atomicity. **Semaphore_tour** variable is used to prevent visitors in a tour to leave before the guide(if it exists). **Guide_thread** variable is used to keep track of the guide_thread. **Cond_tour_end** conditional variable is used for in cases where there is an ongoing tour, hence we need to prevent other visitors to come into the site.

Constructor:

//constructor implementation

```
Tour::Tour(int visitorCount, int guide_required) {
```

```
    visitor_count = visitorCount; //number of visitors in the tour
```

```
    this->has_guide = (guide_required == 1); //check if guide is required
```

```
    current_visitors = 0; //since we have just created the tour there are no visitors yet
```

```
    tour_in_progress = false; //initially no tours have started hence it is false
```

```
    // 1. Input validation
```

```
    if (visitorCount <= 0) {
```

```
        throw std::invalid_argument("visitorCount must be greater than 0");
```

```
    }
```

```
    if (guide_required != 0 && guide_required != 1) {
```

```
        throw std::invalid_argument("Exception caught: An error occurred.");
```

```
    }
```

```
    all_count = has_guide ? visitor_count + 1 : visitor_count; //if there needs to be a guide then the number of people in  
the guide need to be visitor_count + 1 else visitor_count
```

```
    //all_count = visitor_count;
```

```
    //initializing the visitor mutex
```

```
    if (pthread_mutex_init(&visitor_mutex, nullptr) != 0) {
```

```
        throw std::runtime_error("Failed to initialize visitor_mutex");
```

```
    }
```

```
    if (sem_init(&semaphore_tour, 0, 0) != 0) { // Blocked initially
```

```
        pthread_mutex_destroy(&visitor_mutex);
```

```
        throw std::runtime_error("Failed to initialize semaphore_tour");
```

```
    }
```

```
    if (pthread_cond_init(&cond_tour_end, nullptr) != 0) {
```

```
        sem_destroy(&semaphore_tour);
```

```

pthread_mutex_destroy(&visitor_mutex);

throw std::runtime_error("Failed to initialize cond_tour_end");
}

// Initialize guide_thread to a default value for safety
guide_thread = pthread_t(); // This sets guide_thread to an uninitialized thread state
}

```

In the constructor we initialize the all_count variable (depending on whether there is a guide or not we increment it by 1 as mentioned in the homework document). We throw exceptions just in case the input is not properly passed. We initialize the conditional variables and semaphores and initialize the guide_thread variable. If an exception occurs we do a proper roll back to prevent any resource leaks.

Destructor:

```

Tour::~~Tour() {
    pthread_cond_destroy(&cond_tour_end);
    // Destroy the visitor mutex
    pthread_mutex_destroy(&visitor_mutex);

    // Destroy the semaphores
    sem_destroy(&semaphore_tour);
}

```

The destructor logic is quite simple, we just destroy the 3 variables that we created in the constructor, these variables are cond_tour_end, visitor_mutex, semaphore_tour. The aim of this function is to prevent any resource leaks, and ensure a successful termination.

Arrive function:

```

void Tour::arrive() {
    printf("Thread ID: %lu | Status: Arrived at the location.\n", pthread_self());

    pthread_mutex_lock(&visitor_mutex); // we need to use a mutex to prevent race conditions

    while(tour_in_progress) {
        //if we are here it means the there is tour in progress
        //pthread_mutex_unlock(&visitor_mutex); // Unlock before blocking
        //sem_wait(&semaphore_arrive);      // Block until the tour ends
        //pthread_mutex_lock(&visitor_mutex); // Reacquire lock after unblocking
        pthread_cond_wait(&cond_tour_end, &visitor_mutex); // Wait until the tour ends
    }

    //if we are here it means that no tour is in progress
    current_visitors++; //increment the current number of visitors

    if (current_visitors == all_count) {
        //if we are here it means that the tour capacity is full
        if (has_guide) {
            //if we are here it means the tour contains a guide
            guide_thread = pthread_self(); //the last visitor is assigned as a guide
        }

        printf("Thread ID: %lu | Status: There are enough visitors, the tour is starting.\n", pthread_self());
        tour_in_progress = true;

        // Synchronize all visitors for the tour start
        //pthread_barrier_wait(&barrier_tour_start);
        //*****

        //pthread_mutex_unlock(&visitor_mutex);
        // Synchronize all visitors for the tour start (I am not sure about this one, I will check it later on)
        //pthread_barrier_wait(&barrier_tour_start);
    } else {
        //if we are here it means that there are not enough visitors in the tour
        printf("Thread ID: %lu | Status: Only %d visitors inside, starting solo shots.\n", pthread_self(), current_visitors);

        // Release the mutex so the visitor can decide to leave later
    }
}

```

```

        //pthread_mutex_unlock(&visitor_mutex);
    }

    pthread_mutex_unlock(&visitor_mutex); // Unlock mutex
}

```

Before starting we print an appropriate message to the console, and then get a hold of the lock. The aim of the lock is to ensure atomicity. One question that might come into mind is it feels like a coarse grained solution but it is rather not because in the arrive function we need to increment the current_visitors but to ensure that no such thread increments the number of visitors in during a tour is in progress we need to use a lock. Hence the sole purpose of this lock is to ensure atomicity of the increment operation. The reason why we can't release the lock when we increment current_visitors is because we also need to check if current_visitors == all_count and then update the guide if needed. Updating the guide_thread again needs to be atomic hence we would need to get a hold of the lock once again but this would be less efficient than the previous one because there will be overheads. Hence it is more suitable to use a lock at the beginning and release at the end. Now we have a mutex at the beginning, but we have a while loop as well, if this loop fails then the thread will spin hence will slow the code. The solution to this problem is to use a conditional variable, this way when there is an ongoing tour we will block the thread and wait until the tour ends. If the tour has not started yet, the thread continues and increments the current_visitors variable. And then does a check to see if the tour has enough visitors to start. After doing appropriate updates and printing to the console we release the lock for further operations done by other visitors. We have to use a lock mechanism to prevent race conditions and this mechanism needs to be implemented at the beginning so that no threads enter the critical region. Let's examine why this function works correctly. There are 2 scenarios the first one is when there is a tour that is being executed and a visitor tries to enter the site. When the visitor calls the arrive method it will check if tour_in_progress variable is true. Since a tour is being executed currently the visitor will block itself and wait until the tour ends. Before blocking it releases the lock(because of pthread_cond_wait() function) so there is no potential deadlock possibilities. The second scenario is when there is a tour that is being executed currently hence the visitor(thread) can continue, the current_visitors variable is incremented atomically since we have the lock. And then checks if the tour has enough people, if not the thread finishes the arrive function(of course releases the lock before leaving). If the tour has enough people then it handles the logic accordingly and then terminates the arrive function. As we can see there are no possible deadlock issues hence this function is thread safe.

Leave function:

```

void Tour::leave() {

```

```

pthread_mutex_lock(&visitor_mutex); //this mutex is only used for ensuring atomicity of decrement operations,
hence there will be no busy waiting

if (!tour_in_progress) {
    //if we are here it means that the tour is not in progress so the visitors can leave whenever they want
    printf("Thread ID: %lu | Status: My camera ran out of memory while waiting, I am leaving.\n", pthread_self());
    if (current_visitors > 0) {
        current_visitors--; // Decrement the visitor count
    }
    pthread_mutex_unlock(&visitor_mutex); // Unlock and exit
    return; //the visitor has left hence we return from the function
}

if (has_guide && pthread_equal(pthread_self(), guide_thread)) {
    //if we are here it means that this is the guide and the guide finishes the tour
    printf("Thread ID: %lu | Status: Tour guide speaking, the tour is over.\n", pthread_self());
    current_visitors--; // Decrement the visitors count
    sem_post(&semaphore_tour); // Unblocking the waiting visitors (that are in the tour and waiting for the guide to
leave)
} else {
    if (has_guide) {
        //if we are here it means the guide exists in the tour hence we need to wait until the guide leaves
        pthread_mutex_unlock(&visitor_mutex); //unlocking the mutex before blocking
        sem_wait(&semaphore_tour); // Wait for the guide to leave
        pthread_mutex_lock(&visitor_mutex); // Reacquiring the lock once the guide leaves the tour
    }
    //if we are here, it means that the tour had no guide or the visitor is leaving after the guide
    printf("Thread ID: %lu | Status: I am a visitor and I am leaving.\n", pthread_self());
    if (current_visitors > 0) {
        current_visitors--; // Decrement the visitors count
    }
    if (current_visitors > 0) {
        sem_post(&semaphore_tour); // Unblock the next visitor in line
    }
}

if (current_visitors == 0) {

```

```

//if we are here it means the last visitor is leaving hence we announce that new visitors can come
printf("Thread ID: %lu | Status: All visitors have left, the new visitors can come.\n", pthread_self());

tour_in_progress = false; // Reset the tour status

pthread_cond_broadcast(&cond_tour_end);

//sem_post(&semaphore_arrive); // Unblock any visitors waiting to enter
// pthread_cond_broadcast(&cond_tour_end);
}

pthread_mutex_unlock(&visitor_mutex); // Unlock the mutex
}

```

In the leave function again we are using a lock, but as we can see there is no loop. The sole purpose of lock again in this case is to prevent overhead issues. If a tour is not in progress then it means a visitor wants to leave before the tour starts hence we display an appropriate message. And then we decrement current_visitors since this visitor(thread) has left. After decrementing it returns from the leave function(of course it release the lock first to prevent deadlocks). Now here is the crucial part, if there is a guide all the visitors need to wait for the guide to leave the tour(of course if a tour is in progress). To ensure this we use semaphores, because if we were to use a loop then it would cause a busy wait which is not resource friendly. If the current thread is not the guide it calls sem_wait() function so that it blocks itself until the guide leaves. If the visitor is also the guide then it displays and appropriate message and then decrements current_visitors by 1 since it has left the tour. But before terminating the guide needs to call sem_post to wake the next thread, this way it makes sure that the next thread is woken up and it can continue. Once that thread is woken up, that thread leaves the tour as well but before leaving it also wakes another visitor. Basically we are creating a queue for the visitors to exit the tour site one by one, this way both atomicity is ensured and concurrency was not harmed. This implementation is correct because we are using locks to ensure atomicity and semaphores to avoid busy waiting, and also make the ordering. One thing that is very important is that once the last visitor leaves the site it needs to make sure that it wakes up the sleeping visitors that were in the arrive methods. These visitors were in sleep because there was an ongoing tour at their arrival hence they couldn't enter the site. To achieve this we use pthread_cond_broadcast to wake all the sleeping threads. The reason why we do not wake them up one by one is because we want to make sure that we do not give away from concurrency and fairness hence we wake them all up at the same time and they all have equal chance to enter the site.

Conclusion:

Overall I used locks, semaphores, and conditional variables to ensure atomicity, increase concurrency, and prevent busy waitings which can result in unnecessary resource consumption. There is 1 constructor to initialize any needed variables, 1 destructor to ensure no resource leaks occur, and 2 functions: arrive and leave that are implemented to ensure a correct scheduling for the visitors to enter and leave a sight whenever they want.